

LLM 및 CVSS 기반 자동화된 웹 취약점 진단 및 리포팅 파이프라인

Automated DAST & LLM-based Remediation Pipeline



이세혜

202155530 김세한 sehankim@pusan.ac.kr

202355579 전해은 silver807@pusan.ac.kr

202155577 이경윤 kylee0293@gmail.com

교수: 손준영 교수님

제출일: 2026년 7월 17일

부산대학교

목차

1	추진 현황 및 요약	1
1.1	과제 개요	1
1.2	중간보고 시점 달성 현황	1
2	요구조건 및 제약사항 수정사항	2
3	시스템 설계 및 구현 현황	3
3.1	전체 시스템 아키텍처	3
3.2	하이브리드 탐지 파이프라인	4
3.3	프론트엔드·백엔드·인프라 구현	4
4	수행 내용 및 중간 결과 — AI 탐지 모델	6
4.1	학습 데이터 구축 (CVEfixes)	6
4.2	학습 데이터 포맷 (프롬프트·정답·메타)	8
4.3	모델 및 파인튜닝 설정 (Qwen3.5-9B QLoRA)	9
4.4	채점 방법	10
4.5	성능 평가 결과 (test 1,197건)	10
4.6	상용 모델과의 비교 (완전 동일 조건)	11
4.7	외부 벤치마크 검증 (일반화 성능)	12
4.8	개선 실험 (진행 중)	12
5	구성원별 진척도	13
6	갱신된 과제 추진 계획	14
6.1	남은 과업 및 우선순위	14
6.2	개발 일정	14
7	현실적 제약 사항 및 대책	15
8	결론	16

제1장 추진 현황 및 요약

1.1 과제 개요

2025년 확산된 ‘바이브 코딩(vibe coding)’ — 자연어로 AI에게 지시해 코드를 생성하는 방식 — 으로 코드 생산량이 폭발했으나, ‘AI가 짠 코드는 안전할 것’이라는 통념과 달리 보안 결함이 다수 유입되고 있다. 보안분석 기업 Veracode의 2025년 조사(100개 이상 LLM 대상)에서도 생성 코드의 상당수가 취약점을 포함했으며, SK텔레콤 유심 정보 유출 등 실제 사고도 이어지고 있다. 그럼에도 소규모 개발팀은 자체 보안 인력이 없고 상용 정적분석(SAST) 도구는 비싸다.

본 과제는 GitHub 저장소 주소나 웹사이트 주소만 입력하면 코드를 외부로 전송하지 않고 자체 서버에서 취약점을 진단하는 SaaS다.

핵심은 (1) 보안 취약점 탐지에 특화되도록 자체 파인튜닝한 LLM과 (2) 코드 그래프(taint) 기반 정적 분석을 결합한 하이브리드 탐지 엔진을 자체 인프라에서 운영하는 것이다. 본 중간보고서는 이 중 AI 탐지 모델을 처음부터 재구축하고, 상용 최상위 모델 및 외부 공개 벤치마크와 비교 검증한 결과를 중심으로 서술한다.

1.2 중간보고 시점 달성 현황

중간보고 시점 현재, AI 탐지 모델의 학습·검증을 완료하였고, 프론트엔드·백엔드·인프라의 핵심 경로(회원가입→인증→스캔 요청→결과 조회)를 실제 배포 환경에서 동작 확인하였다.

표 1.1: 중간보고 시점 달성 현황

항목	달성	비고
AI 탐지 모델 학습 (Qwen3.5-9B QLoRA)	완료	CVEfixes 11,601건, 시간+CVE그룹 분할
내부 test 성능 평가 (1,197건)	완료	재현율 79.7% / 오답률 15.7% / F1 80.5
상용 모델 비교 (Claude·Grok)	완료	완전 동일 조건, 오답률 우위 확인
외부 벤치마크 검증 (PrimeVul·CleanVul)	완료	누수 제거 후 일반화 성능 정량화
AI 모델 서버 연결 (RunPod)	진행	채점 검증 완료, 서버 배선 작업 중
프론트엔드 16종 화면 / 백엔드 18개 API	완료	Vercel·EC2 배포
코드 그래프(taint) 정적 분석 결합	진행	웹 계열 CWE 보장 규칙 설계

제2장 요구조건 및 제약사항 수정사항

착수 대비 변경·구체화된 내용을 정리한다.

- 1) **AI 모델 재구축 (레거시 앙상블 → 단일 파인튜닝)**: 기존 다단계 앙상블 구성을 걷어내고, 학습 데이터 파이프라인부터 평가까지 이해 가능한 단일 파인튜닝 모델(Qwen3.5-9B QLoRA)로 재출발하였다. 데이터 누수 차단·시간 분할·상용/외부 벤치마크 비교를 표준화해 성능을 재현·검증 가능하게 했다.
- 2) **인프라 구조 변경 (Railway → AWS EC2)**: 초기 Railway PaaS에서 AWS EC2로 이전하고, 상시 가동이 필요한 백엔드와 스캔 시에만 필요한 무거운 연산(LLM 추론·DAST)을 분리하였다.
- 3) **모델 서빙 구조 변경 (온프레미스 → RunPod 서버리스 GPU)**: CPU 직접 서빙은 스캔 1건당 추론이 GPU 대비 4~5배 느려, on-demand 특성을 살려 LLM 추론만 RunPod 서버리스 GPU로 분리하였다.
- 4) **DAST(ZAP) 인스턴스 분리**: Active Scan 중 서버 전체가 응답 불능에 빠지는 문제를 해결하기 위해 OWASP ZAP을 별도 EC2로 격리하고, API를 내부 전용으로 제한해 임의 스캔·SSRF를 차단하였다.
- 5) **인증 구조 정비**: 프론트에만 있던 목업 인증을 백엔드 실제 회원가입/로그인 API(BCrypt+JWT)로 대체하였다. GitHub OAuth와 식별자를 통일하였다.

3.1 전체 시스템 아키텍처

```

graph LR
    User[사용자  
(브라우저)] --> Frontend[프론트엔드 Vercel  
React · TypeScript  
16중 화면 · 스캔 3모드]
    Frontend -- HTTPS --> Backend[백엔드 t3.medium  
Spring Boot · Java 17  
OAuth2 + JWT  
18개 API · 상시 가동]
    Frontend -- REST API --> Backend
    Backend --> PG[PostgreSQL  
Flyway 스키마 관리]
    Backend --> DAST[DAST EC2 c5.large  
OWASP ZAP  
:8090 내부 전용]
    Backend -- 웹 DAST 스캔 --> DAST
    Backend -- SAST 추론 요청 --> ModelServer[model-server  
Python · FastAPI  
:8100 내부 전용]
    ModelServer -- GPU 추론 --> RunPod[RunPod Serverless  
GPU  
파인튜닝 LLM 추론]
    GitHub[GitHub  
레포 연동 · OAuth] -- 레포 클론 · OAuth --> Backend
    subgraph AWS [AWS]
        Backend
        PG
        DAST
        ModelServer
    end
    style AWS fill:none,stroke-dasharray: 5 5
    style AWS_label fill:none,stroke:none
    AWS_label[AWS 외부 노출: 백엔드 HTTPS만]
  
```

사용자 (브라우저) → 프론트엔드 (Vercel) (React · TypeScript, 16중 화면 · 스캔 3모드) → 백엔드 (t3.medium) (Spring Boot · Java 17, OAuth2 + JWT, 18개 API · 상시 가동)

백엔드 (t3.medium) → PostgreSQL (Flyway 스키마 관리)

백엔드 (t3.medium) → DAST EC2 (c5.large) (OWASP ZAP, :8090 내부 전용) (웹 DAST 스캔)

백엔드 (t3.medium) → model-server (Python · FastAPI, :8100 내부 전용) (SAST 추론 요청) → RunPod Serverless (GPU, 파인튜닝 LLM 추론) (GPU 추론)

GitHub (레포 연동 · OAuth) → 백엔드 (t3.medium) (레포 클론 · OAuth)

AWS (외부 노출: 백엔드 HTTPS만)

표 3.1: 구성 요소별 역할

3

3.2 하이브리드 탐지 파이프라인

SAST 스캔은 두 경로를 병렬로 수행한다. ① 파인튜닝 LLM이 코드 조각을 받아 취약여부·CWE·CVSS·원인을 4줄로 판정하고, ② 코드 그래프(taint) 정적 분석이 데이터 흐름을 분석해 웹 계열 취약점을 보강한다. 두 결과를 결합해 CWE·CVSS·원인·해결방법을 포함한 취약점 목록으로 정리한다(그림 3.2).

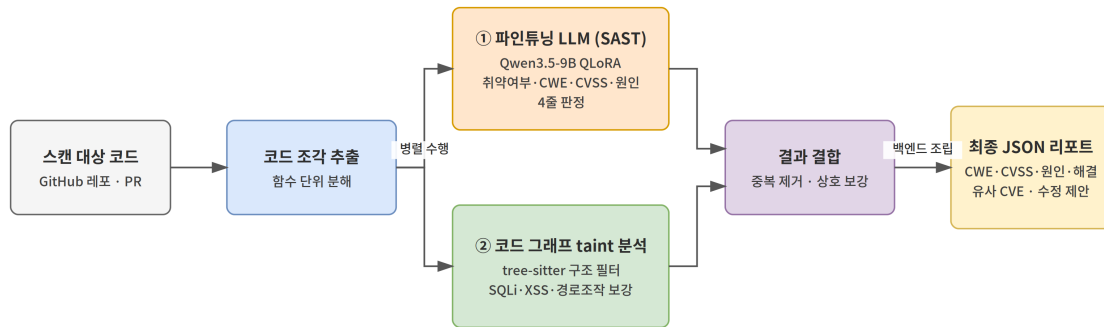


그림 3.2: 하이브리드 탐지 파이프라인

설계 요지

- LLM은 넓은 언어·취약 유형을 빠르게 커버하고, taint 그래프는 데이터 흐름이 명확한 웹 계열(SQLi·XSS·경로조작)에서 LLM이 놓친 건을 보강한다.
- 모델은 4줄만 출력하고, 백엔드가 유사 CVE 사례·수정 제안을 조립해 최종 JSON 리포트를 구성한다.

3.3 프론트엔드·백엔드·인프라 구현

- **프론트엔드** (담당: 김세한): React, TypeScript, FSD 아키텍처로 랜딩·회원가입·대시보드·스캔 요청·리포트 등 16종 화면을 구현하고 Vercel에 배포하였다. 스캔은 웹(DAST), 레포(SAST), PR 3모드로 제공한다.
- **백엔드** (담당: 전해은): Spring Boot (Java 17), Spring Security (OAuth2+JWT), PostgreSQL 기반. Auth, GitHub, Scan, Verify, Report, Vulnerability 등 9개 Controller, 18개 엔드포인트를 구현하였고, ZAP 연동은 non-blocking 풀러로 처리한다.
- **인프라·DAST** (담당: 전해은): Railway에서 AWS EC2로 마이그레이션 완료, Docker Compose 운영 및 Flyway 스키마 관리. 백엔드 EC2만 외부에 노출하고 ZAP·model-

server는 내부 전용으로 운영한다.

- 코드 그래프 (담당: 이경윤): tree-sitter 기반 구조 필터(선언부·보일러플레이트 자동 SAFE 처리)와 다국어 taint 분석(Java/JS·TS/Python) 결합 규칙을 설계·구현하였다.

제4장 수행 내용 및 중간 결과 — AI 탐지 모델

본 장은 이번 중간보고의 핵심으로, AI 탐지 모델을 처음부터 재구축한 과정과 성능을 다룬다. 학습 데이터의 구조·전처리·분할, 모델에 던지는 프롬프트/정답(completion) 형식, 파인튜닝 설정, 채점 방식, 그리고 상용 모델·외부 벤치마크와의 비교 결과를 순서대로 서술한다. 모든 평가는 모델이 학습에서 보지 못한 데이터로 수행하였다. 전체 흐름은 그림 4.1과 같다.

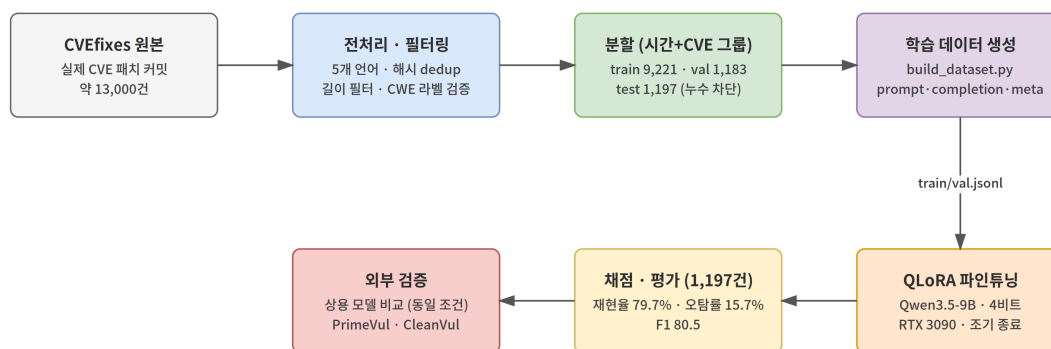


그림 4.1: AI 탐지 모델 학습·평가 파이프라인

4.1 학습 데이터 구축 (CVEfixes)

원본은 실제 CVE 패치 커밋을 모은 공개 데이터셋 CVEfixes(약 13,000건)다. 각 행은 하나의 취약점 수정 커밋으로, 패치 이전 코드(vulnerable_code)와 이후 코드(fixed_code)를 함께 담고 있다. 이 중 모델 학습에 실제로 쓰는 필드는 다음과 같다.

cvss2_base_score(구버전 점수), version_tag·repo 규모 정보, diff_stats, security_keywords(대부분 빈 배열) 등은 취약점 학습과 무관하거나 신뢰도가 낮아 제외하였다.

전처리 (필터링)

- 1) 언어 필터 — 주요 5개 언어(Python, JavaScript/TypeScript, Java, PHP, C/C++)만 남긴다.
- 2) 코드 정규화 해시 dedup — 공백·주석을 정규화한 해시로 중복 코드를 제거한다. 두

표 4.1: CVEfixes에서 학습에 사용한 필드

필드	의미 · 용도
vulnerable_code	패치 이전의 취약한 코드 → 라벨 ‘취약’ 샘플
fixed_code	패치 이후의 수정된 코드 → 라벨 ‘안전’ 샘플
cwe_id / cwe_name	취약점 유형(예: CWE-89 = SQL Injection) — 모델이 출력할 정답
cvss3_base_score / severity	심각도 점수(0~10)와 등급 — 모델이 출력할 두 번째 정답
cve_id	CVE 고유번호 — train/test 분할의 기준 키 (같은 CVE가 갈라지면 누수)
published_date	CVE 공개일 — 시간 기준 분할에 사용
cve_description / cwe_description	취약점 원인의 자연어 설명 — REASON 생성 재료

CVE에 같은 함수가 있을 수 있어, 분할 전에 제거해야 test/train으로 갈라지는 누수를 막는다.

- 3) 길이 필터 — 너무 짧거나(min 40자) 너무 긴(max 4,096토큰) 코드를 제외한다.
- 4) CWE 라벨 없는 행 제외 — cwe_id가 없는 행(NVD-CWE-noinfo 등 1,020건)은 취약 정답을 만들 수 없어 제외한다.

데이터 분할 (시간 + CVE 그룹) published_date로 정렬해 최신 10%를 test로 분리(시간 분할)하고, 나머지에서 cve_id 그룹 단위로 train:val = 8:1로 나눈다. 같은 CVE의 취약본·수정본이 train과 test로 갈라지면 누수가 되므로, 반드시 CVE 단위로 묶어서 분할한다.

표 4.2: 최종 데이터셋 규모 (총 11,601건)

분할	샘플 수	서로 다른 CVE	구성 방법
train	9,221	5,132	test·val을 뺀 나머지 CVE 그룹
val	1,183	641	train에서 무작위 10% 분리
test	1,197	641	최신 10% CVE (2023-07~2024-07 신규)

- **라벨 비율:** 취약 44 : 안전 56. (test 재현율이 유독 낮으면 라벨 불균형을 의심하고 안전 샘플 다운샘플링으로 재실험할 여지를 남겨둠)
- **언어별 test 표본:** PHP 391 / C·C++ 297 / Python 221 / JS·TS 219 / Java 69 (Java는 표본 부족으로 참고치, 추후 보강 수집 1순위)
- **결측 severity 처리:** severity가 nan인 경우가 많아, cvss3_base_score가 있으면 점수로 등급을 도출(9.0+ CRITICAL / 7.0+ HIGH / 4.0+ MEDIUM / 미만 LOW)하고, 둘 다 없으면 SEVERITY·CVSS 줄을 UNKNOWN으로 둔다.

4.2 학습 데이터 포맷 (프롬프트 · 정답 · 메타)

필터링된 데이터를 모델이 학습할 수 있는 형식으로 2차 가공한다(build_dataset.py → train/-val/test.jsonl). 각 줄은 프롬프트(prompt)·정답(completion)·메타(meta)로 구성된다. 프롬프트에 출력 서식을 명시해 파싱을 안정화하고, 상용 모델과도 완전히 같은 프롬프트로 공정 비교가 가능하게 했다.

프롬프트 템플릿 (입력)

Analyze the following {language} code for security vulnerabilities.

```
```{language}
{code} <- 취약 샘플이면 vulnerable_code, 안전 샘플이면 fixed_code
```
```

Respond in exactly this format:

VULNERABILITY: <CWE-id (CWE name)> or NONE

SEVERITY: <CRITICAL|HIGH|MEDIUM|LOW|UNKNOWN> or NONE

CVSS: <score 0.0-10.0> or 0.0

REASON: <one-line explanation> or NONE

정답(completion) 생성 규칙 (출력) 모델은 위 서식에 맞춰 4줄만 출력하도록 학습한다. 취약/안전 샘플에 따라 정답을 다르게 만든다.

- **취약 샘플:** VULNERABILITY 줄에 cwe_id (cwe_name)을 출력한다. SEVERITY는 ① cvss3_base_score를 공식 구간표로 도출(원본 severity 필드는 v2 기준이 섞여 CVSS 줄과 모순되므로 우선순위를 낮춤) → ② cvss3 없으면 원본 severity → ③ 둘 다 없으면 UNKNOWN. CVSS는 cvss3_base_score(결측 시 UNKNOWN). REASON은 cve_description에서 원인 서술 문장을 추출('allows / due to / fails to / via' 등 패턴)하고, 실패 시 cve_description으로 풀백한다.
- **안전 샘플:** 네 줄 모두 비어 있음을 명시 — VULNERABILITY: NONE / SEVERITY: NONE / CVSS: 0.0 / REASON: NONE.

메타(meta) 구조 — 채점 정답지 meta는 학습에는 쓰이지 않고, 언어별·CWE별 채점 리포트를 만들 때 정답지로 쓰인다.

실제 학습 데이터 예시 (JavaScript, CWE-22 경로조작)

```
{"prompt": "Analyze the following JavaScript code for security\nvulnerabilities.\n\n```\nJavaScript\n취약{ 코드}\n```\n\nRespond in\nexactly this format:\n\nVULNERABILITY: ...\nSEVERITY: ...\nCVSS: ...\nREASON: ...",\n"completion": "VULNERABILITY: CWE-22 (Improper Limitation of a Pathname
```

표 4.3: meta 필드 구조

| 필드 | 의미 |
|-----------------------|--|
| cve_id | 출처 CVE (분할 기준 키였던 것) |
| language / lang_group | 원본 언어 라벨 / 리포트용 5개 언어 그룹 (JS·TS, C·C++ 묶음) |
| cwe_id | 취약 유형 (채점 정답) |
| severity | 심각도 등급 (채점 정답) |
| published_date | CVE 공개일 (시간 분할 근거) |
| label | vuln(취약) / safe(안전) — 재현율·오탐률 채점 정답 |

```
to a Restricted Directory ('Path Traversal'))\nSEVERITY: HIGH\nCVSS: 7.5\n
REASON: The product uses external input to construct a pathname ...",
"meta": {"cve_id": "CVE-2020-7763", "language": "JavaScript",
"lang_group": "JS/TS", "cwe_id": "CWE-22", "severity": "HIGH",
"published_date": "2020-11-05T14:15Z", "label": "vuln"}}
```

4.3 모델 및 파인튜닝 설정 (Qwen3.5-9B QLoRA)

베이스 모델은 Qwen3.5-9B instruct다. 더 큰 Qwen3-Coder-30B는 비용·필요 GPU·VRAM 문제로 제외하였다. 학습은 4비트 QLoRA로, 동결된 베이스를 4비트로 압축(18GB→약 6GB)해 RTX 3090 24GB 단일 GPU에서 파인튜닝했다. LoRA는 트랜스포머의 모든 선형층(attention 4개 + MLP 3개)에 어댑터를 부착한다. 학습 프레임워크는 Unsloth를 사용하였다.

표 4.4: QLoRA 파인튜닝 설정값

| 설정 | 값 | 근거 |
|---------------------|----------|--------------------------------------|
| LoRA rank / alpha | 16 / 32 | ~1만 건 규모의 통용 기본값, alpha는 관례상 rank×2 |
| learning rate | 2e-4 | QLoRA 표준값 |
| max epochs | 3 | 상한 — early stopping이 보통 먼저 걸림 |
| early stop patience | 3 | val loss 3회 연속 미개선 시 중단, 최적 체크포인트 채택 |
| max sequence length | 4,096 | 데이터 길이 필터와 동일 |
| 유효 배치 | 16 (2×8) | 24GB VRAM 기준 device당 2 × 누적 8 |
| seed | 42 | 재현성 |

학습 흐름은 ① 베이스를 4비트로 로드 → ② LoRA 어댑터 부착(베이스 동결, 어댑터만 학습) → ③ prompt/completion을 Qwen 채팅 템플릿으로 감싸고 → ④ completion 부분만 loss를 계산(프롬프트 베끼기에 학습을 낭비하지 않도록) → ⑤ val loss를 감시해 조기 종료하는 순서다. 학습 산출물은 어댑터 본체(adapter_model.safetensors, 116MB)와 토큰나이저 세트다.

```
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=32, lora_dropout=0.0,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    use_gradient_checkpointing="unsloth", random_state=42)
```

4.4 채점 방법

학습된 어댑터로 test.jsonl을 추론하고, 생성된 4줄을 정규식으로 파싱해 (취약여부·CWE·SEVERITY)를 추출한 뒤 meta의 정답과 대조한다.

채점 대상과 제외는 다음과 같다.

- **채점 대상 (기계 채점):** ① 취약 여부(vuln/safe) → 재현율·오탐률의 재료, ② CWE 유형 일치, ③ SEVERITY 등급 일치.
- **채점 제외 (표시용):** CVSS 숫자(화면 표시용, severity와 중복이나 사용자 체감상 유지), REASON(설명용).

표 4.5: 채점 지표 정의

| 지표 | 정의 |
|----------------|----------------------------|
| 재현율(recall) | 잡아낸 취약 / 전체 취약(552) |
| 오탐률(FPR) | 취약이라 오판한 안전 / 전체 안전(645) |
| 정밀도(precision) | 잡은 것 중 진짜 / 잡은 것 전체 |
| F1 | 재현율·정밀도의 조화평균 (둘 다 좋아야 높음) |
| CWE/SEV 일치율 | 취약 정답 건 중 유형/등급까지 맞춘 비율 |

실행환경은 RunPod RTX 3090 24GB이며, 1,197건 채점에 약 44분(건당 ~2.2초, 배치 4). 하이브리드 linear-attention 모델이라 flash-linear-attention·causal-conv1d 커널 설치 전에는 토큰당 ~18초로 느렸고, 커널 설치 후 건당 ~3초로 단축되었다. 1,197건 중 파싱 실패는 3건으로, 4줄 포맷 학습이 잘 이루어졌음을 보여준다.

4.5 성능 평가 결과 (test 1,197건)

학습에서 보지 못한 신규 CVE 1,197건 채점 결과다. **전체: 재현율 79.7% · 오탐률 15.7% · F1 80.5** (파싱 실패 3건).

CWE별에서 나타난 뚜렷한 패턴

표 4.6: 언어별 성능 (재현율 / 오탐률 / F1)

| | 전체 | C/C++ | PHP | Python | JS/TS | Java* |
|-----|-------|-------|-------|--------|-------|-------|
| 재현율 | 79.7% | 79.8% | 78.1% | 79.3% | 80.2% | 86.8% |
| 오탐률 | 15.7% | 28.3% | 7.5% | 10.4% | 19.5% | 6.5% |
| F1 | 80.5 | 72.8 | 83.5 | 83.2 | 79.8 | 90.4 |

- 웹 계열은 유형까지 잘 맞춘다(경로조작 재현 93%·유형일치 67%, SQLi 82%·68%).
- 반면 C/C++ 메모리 계열은 ‘취약한 것은 아는데 어떤 메모리 버그인지 혼동’한다. CWE-120/787/284는 재현율 90%+인데 유형일치 0% — 서로 비슷한 메모리 CWE로 바뀌 답한다. CWE 전체 일치율 27.9%가 낮아 보이는 것은 대부분 이 혼동 탓이다.
- C/C++ 오탐률(28.3%)과 함께 이 메모리 CWE 혼동이 다음 개선 타깃으로 명확해졌다. (Java는 표본 69건으로 참고치)

4.6 상용 모델과의 비교 (완전 동일 조건)

같은 test 1,197건·같은 프롬프트로 상용 최상위 모델(Claude Sonnet 5)·Grok-4와 비교했다. 차이는 ‘누가 답을 생성했는가’뿐이다.

표 4.7: 상용 모델 비교

| 모델 | 재현율 | 오탐률 | F1 | 정밀도 |
|-----------------|-------|-------|------|-------|
| 자체 파인튜닝 모델 | 79.7% | 15.7% | 80.5 | 81.3% |
| Claude Sonnet 5 | 82.6% | 79.5% | 60.0 | 47.1% |
| Grok-4 | 57.2% | 46.7% | 54.1 | 51.2% |

관찰 — 상용 모델의 오탐은 왜 발생하는가 자체 모델은 재현율에서 상용 모델과 대등하면서도 오탐률이 압도적으로 낮다(15.7% vs Claude 79.5%).

상용 범용 모델의 과잉 경보에는 두 가지 구조적 원인이 있다. ① **프롬프트 프레이밍** — ‘취약점을 찾아달라’는 지시는 모델을 무언가를 찾아내는 방향으로 유도하며, 안전을 놓치는 위험보다 과잉 경보를 선호하도록 조율된 범용 어시스턴트의 특성과 맞물린다. ② **맥락 부재** — 함수 조각만으로는 호출부의 입력 검증 여부를 알 수 없어, 검증이 보이지 않으면 ‘취약 가능’으로 답하는 경향이 있다.

이 과잉 경보가 ‘자체 라벨의 한계’가 아니라 모델 자체의 특성임은 다음 절의 외부 벤치마크에서 입증된다. 라벨을 정제한 CleanVul에서도 오탐률 79.6%로 동일 재현된다. 보안 스캐너의 실사용 병목이 오탐임을 감안하면, **오탐 억제가 특화 모델의 결정적 이점**이다.

4.7 외부 벤치마크 검증 (일반화 성능)

‘자체 데이터로 자체 모델을 평가했다’는 한계를 넘기 위해, 학습 분포 밖의 공개 벤치마크 2종에서 동일 조건으로 재검증했다. 학습셋과 겹치는 표본은 CVE ID·정규화 코드 해시 두 기준으로 제거(누수 차단)하였다.

- **PrimeVul** (arXiv:2403.18624): C/C++ 전용, 취약/패치 쌍 단위 채점이 공식 프로토콜인 벤치마크. 누수 제거 후 180쌍으로 평가.
- **CleanVul** (arXiv:2411.17274): 라벨 노이즈를 걸러낸 고신뢰(~97%) 다언어 세트. 누수 제거 후 5,790쌍으로 평가.

표 4.8: 외부 벤치마크 검증 (오탐률 · 쌍 정답률)

| | 내부 test | PrimeVul | CleanVul |
|--------------|---------|----------|----------|
| 자체 모델 오탐률 | 15.7% | 10.6% | 21.7% |
| Claude 오탐률 | 79.5% | 82.2% | 79.6% |
| 자체 모델 쌍 정답률 | — | 3.3% | 16.1% |
| Claude 쌍 정답률 | — | 2.2% | 7.6% |

성과 두 가지 · 한계 하나

- ① 라벨을 정제한 CleanVul에서도 Claude 오탐률이 79.6%로 내부 test(79.5%)와 동일하게 재현되어, 상용 모델의 오탐이 ‘자체 라벨의 한계’가 아니라 모델 자체의 과잉 경보 특성임이 외부 데이터로 입증됐다.
- ② 취약/패치를 실제로 구분하는 능력을 재는 쌍 정답률에서 자체 모델이 두 벤치마크 모두 상용 모델을 앞섰다.
- **한계:** 학습 분포 밖 재현율(PrimeVul 12.8%, CleanVul C/C++ 15.4%)은 명확한 개선 과제로, 교차 분포 일반화를 다음 목표로 설정했다.

4.8 개선 실험 (진행 중)

교차 분포 재현율을 끌어올리기 위해 학습 데이터를 확장하는 실험을 진행하였다. PrimeVul 학습셋과 CleanVul을 편입하고, 오탐이 심한 JS/TS 쌍은 오버샘플링하였다. 그러나 학습·평가 분할 과정에서 라벨 없는 안전 샘플이 검증셋(val)에 쏠려 조기 종료 판단을 왜곡하는 이슈를 확인하였고, 현재 이를 바로잡아 재설계 중이다. 안정적인 현행 모델을 배포 기준으로 유지하면서, 개선판을 별도로 준비한다.

제5장 구성원별 진척도

표 5.1: 구성원별 역할 및 진척

| 팀원 | 작업 영역 | 세부 진척 (상태) |
|-----|------------------|---|
| 김세한 | AI 모델 (파인튜닝)·프론트 | 학습 데이터 구축·QLoRA 파인튜닝 (완료), 상용·외부 벤치마크 비교 검증 (완료), 데이터 확장 개선 실험 (진행), React 16종 화면 (완료) |
| 이경운 | AI 모델 (코드 그래프) | tree-sitter 구조 필터·다국어 taint 분석 결합 규칙 설계·구현 (완료), LLM-그래프 결합 규칙 (진행) |
| 전혜은 | 백엔드·인프라·DAST | 9개 Controller·18개 API (완료), 이메일+GitHub OAuth 인증 (완료), Railway→AWS 마이그레이션·ZAP 연동 (완료), 결제·사용량 게이트 (예정) |

제6장 갱신된 과제 추진 계획

6.1 남은 과업 및 우선순위

- 1) **AI 모델 서빙 연결** — 학습된 어댑터를 RunPod 서버리스 GPU에 올려 백엔드↔model-server↔GPU 추론 경로를 실동작 연결.
- 2) **교차 분포 일반화 개선** — 데이터 확장 실험의 분할 이슈를 바로잡아 C/C++·JS/TS 약점을 겨냥한 재학습.
- 3) **Neo4j taint 결합** — 코드 그래프의 taint 경로를 탐지 신호로 결합해 웹 계열 재현을 보강, Qdrant 유사 CVE 기반 수정 제안 연동.
- 4) **결제·사용량 게이트** — 플랜별 SAST(LOC)·DAST(횟수) 한도 게이트와 PG 결제 연동.

6.2 개발 일정

표 6.1: 향후 개발 일정

| 과업 | 기간 | 산출물 |
|--------------------------|-------|-------------|
| AI 모델 서빙 연결 (RunPod) | 7월~8월 | 실동작 추론 경로 |
| 모델 개선 재학습 (데이터 확장) | 8월 | 교차분포 성능 개선판 |
| Neo4j taint 결합·유사 CVE 연동 | 8월~9월 | 하이브리드 리포트 |
| 결제·사용량 게이트·통합 테스트 | 9월 | 베타 통합 데모 |
| 최종 검증 및 보고서 | 9월 | 최종보고서 |

제7장 현실적 제약 사항 및 대책

- 1) **교차 분포 재현율** — 학습 분포(CVEfixes) 밖에서 재현율이 하락한다. 대책: Prime-Vul·CleanVul 편입 데이터 확장과 taint 그래프 결합으로 제품 레벨에서 보강.
- 2) **C/C++ 메모리 계열 CWE 혼동** — 취약 여부는 잡지만 세부 메모리 CWE를 혼동한다. 대책: 해당 언어·CWE 표본 보강.
- 3) **GPU 비용** — 상시 GPU는 비용 부담이 커, on-demand 특성에 맞춘 RunPod 서버리스로 스캔 시에만 과금되게 통제.
- 4) **코드 민감성** — 코드 비전송 원칙을 유지하며 취약점 탐지에 쓰는 LLM·벡터DB를 전부 자체 인프라에서 운영, 외부 상용 API로 코드를 보내지 않는다.

제8장 결론

중간보고 시점에서 AI 탐지 모델을 처음부터 재구축하고, 내부 test에서 재현율 79.7%·오탐률 15.7%·F1 80.5를 달성하였다. 상용 최상위 모델(Claude Sonnet 5)과 완전 동일 조건으로 비교한 결과, 재현율은 대등하면서 오탐률은 15.7% 대 79.5%로 압도적 우위를 보였다.

나아가 학습 분포 밖의 공개 벤치마크(PrimeVul·CleanVul)에서도 상용 모델의 과잉 경보가 재현됨을 확인하여, 오탐 억제가 특화 모델의 구조적 강점임을 외부 데이터로 입증하였다. 남은 과제인 교차 분포 일반화는 데이터 확장과 taint 그래프 결합으로 개선하며, 최종보고에서는 모델 서빙 연결과 제품 통합을 완성한다.

Bibliography

- [1] Qwen Team, “Qwen2.5-Coder Technical Report,” 2024.
- [2] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” NeurIPS, 2023.
- [3] Y. Ding et al., “Vulnerability Detection with Code Language Models: How Far Are We? (PrimeVul),” arXiv:2403.18624, 2024.
- [4] Y. Li et al., “CleanVul: Automatic Function-Level Vulnerability Detection in Code Commits,” arXiv:2411.17274, 2024.
- [5] G. Bhandari et al., “CVEfixes: Automated Collection of Vulnerabilities and Their Fixes,” 2021.
- [6] F. Yamaguchi et al., “Modeling and Discovering Vulnerabilities with Code Property Graphs,” IEEE S&P, 2014.
- [7] OWASP, “Zed Attack Proxy (ZAP) Documentation.”
- [8] Veracode, “2025 GenAI Code Security Report,” 2025.